

Araby CodeAI - Architecture

1. Introduction

The **Araby CodeAI Platform** is a simplified AI-powered coding workspace designed to unify project management, real-time code editing, and intelligent AI-assisted development within a single interface. The platform enables users to create, manage, and interact with coding projects across multiple isolated environments, including JavaScript, Python, and Website Development workspaces.

From an architectural standpoint, the system follows a **modular monolith design pattern**, where core business domains such as authentication, project management, file handling, and AI services are logically separated into well-defined modules within a single deployable backend. This approach enables rapid product iteration while maintaining clear boundaries for future scalability and service decomposition.

The platform integrates modern web technologies, including a **React-based frontend**, a **FastAPI backend**, and **MongoDB for persistent storage**, complemented by **Redis for caching and session control**. Additionally, the system incorporates an AI pipeline capable of understanding project context, analyzing code, and generating structured file modifications through intelligent prompt orchestration.

The runtime architecture is designed around a streamlined request flow: the client application communicates through a Nginx gateway to backend APIs and WebSocket services, which coordinate with the database and AI layer to deliver real-time, context-aware responses. Observability is enabled using OpenTelemetry-compatible tracing systems, ensuring visibility into system performance and AI interactions.

This architecture prioritizes:

- **Developer experience** through an integrated coding workspace
- **AI-driven productivity** via contextual code understanding
- **Scalability readiness** through modular service boundaries
- **Real-time interaction** using WebSocket-based streaming

2. System Overview

2.1 Problem Statement

Modern software development workflows are inherently fragmented. Developers are required to continuously switch between multiple disconnected tools, each responsible for a specific aspect of the development lifecycle. These tools typically include:

- A code editor for writing and modifying source code
- A file manager for organizing project structures
- A terminal for executing commands and running applications
- An AI assistant for generating or explaining code
- A web browser for previewing frontend applications
- Separate systems for managing projects, version history, and collaboration

This fragmentation introduces several inefficiencies:

- Increased cognitive load due to constant context switching
- Lack of synchronization between tools
- Reduced developer productivity
- Higher likelihood of inconsistencies across project components

Additionally, most general-purpose AI chat systems operate in a **stateless manner**, meaning they generate responses based only on the current user prompt without awareness of the broader project context. As a result, these systems fail to understand:

- Existing files and directory structures
- Import relationships and dependencies
- Coding conventions already used in the project
- Previously implemented functionality
- The currently active file or selected code segment
- Cross-file dependencies and side effects
- Whether a requested change introduces breaking issues

This limitation leads to several practical problems:

- Generation of redundant or duplicate files
- Incorrect or missing imports
- Partial or incomplete implementations
- Code that does not integrate with the existing project

- Increased manual effort to fix AI-generated output

The Araby CodeAI Platform addresses these challenges by providing a **unified AI-powered coding workspace** where:

- Project management, file editing, AI interaction, and preview capabilities coexist
- The AI assistant is fully aware of the project structure and context
- Users can perform controlled and recoverable file operations
- AI responses are grounded in actual project data rather than isolated prompts

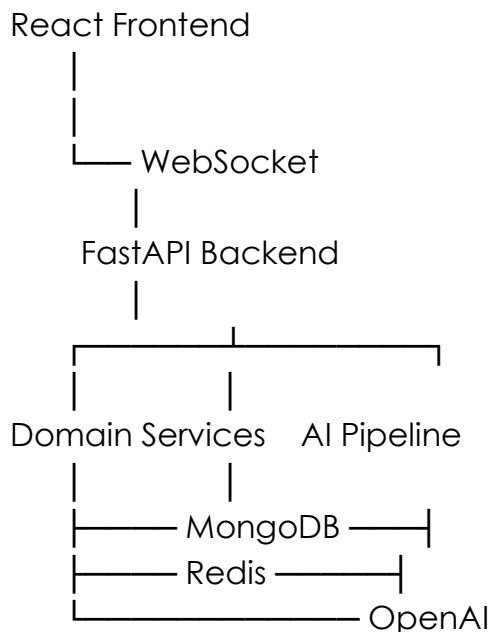
The goal is not to embed a chatbot beside a code editor, but to create an intelligent development environment where AI acts as an integrated development partner capable of reasoning about the entire project state.

2.2 Solution Overview

The system is designed as a layered architecture consisting of four primary components:

- Frontend (React-based user interface)
- Backend (FastAPI-based application server)
- Data Layer (MongoDB and Redis)
- AI Layer (context-aware processing and model interaction)

High-Level Flow



Frontend

The frontend is implemented as a single-page application (SPA) using React and TypeScript. It serves as the primary interaction layer between the user and the system.

Key responsibilities include:

- User authentication and session handling
- Workspace selection (JavaScript, Python, Website Builder)
- Project dashboards for managing user projects
- File explorer for navigating project structures
- Monaco-based code editor for professional editing experience
- AI chat panel for interacting with the assistant
- Live preview environment for website rendering
- Controls for downloading projects and undoing AI changes

State management is handled using:

- TanStack Query for server-side data fetching and caching
- Zustand for client-side state such as authentication, open tabs, and UI state

The frontend communicates with the backend using:

- APIs for standard operations (CRUD, authentication, file management)
- WebSockets for real-time streaming of AI responses

Backend

The backend is implemented using **FastAPI**, chosen for its asynchronous capabilities and high performance.

It follows a strict layered architecture:

Router → Service → Repository → Database

- **Routers** define API endpoints and WebSocket handlers
- **Services** implement business logic and enforce ownership rules
- **Repositories** abstract database interactions
- **Pydantic schemas** ensure request/response validation

The backend is structured as a modular monolith, meaning:

- All modules are deployed as a single application
- Internal boundaries between domains are clearly defined
- Future migration to microservices is possible without major redesign

The backend also contains the AI pipeline, which is responsible for:

- Context collection
- Model selection and routing
- Prompt construction
- Response parsing
- File modification execution
- Preview validation and repair

Data Layer

MongoDB

MongoDB is used as the primary database for storing durable application data. It supports flexible document-based storage, which aligns well with project and file structures.

Stored entities include:

- Users
- Projects
- Folders
- Files
- Chat sessions
- Chat messages
- Refresh tokens

Redis (In-Memory Store)

Redis is used for performance optimization and temporary data storage, including:

- Login rate-limiting counters
- Revoked JWT tokens (blacklist)
- Cached project file trees
- Recently edited file paths
- Short-lived AI context metadata

This separation ensures that:

- MongoDB handles long-term persistence
- Redis handles high-speed, transient operations

AI Layer

The AI system is deeply integrated into the platform rather than functioning as a standalone chatbot.

For every user request, the backend constructs a rich contextual representation that may include:

- Workspace type
- Project metadata
- Complete file path inventory
- Relevant file contents
- Import relationships
- Current file being edited
- Open editor tabs
- Recently modified files
- Selected code snippets
- Previous chat history
- Current user instruction

This context is passed to a language model, which generates a response that includes:

- Explanatory text
- Structured file operations (create, update, delete)

The backend then:

- Parses the response
- Validates file operations
- Applies changes through the file service
- Stores the result for undo and audit purposes

2.3 Key Features

2.3.1 Multi-Workspace System

The platform supports three specialized workspaces, each optimized for a specific development context.

JavaScript Workspace

Designed for modern JavaScript and TypeScript development.

Supported file types:

- .js
- .jsx
- .ts

- .tsx

The AI is instructed to:

- Maintain existing module systems
- Respect import structures
- Follow project naming conventions
- Avoid introducing conflicting patterns

Python Workspace

Designed for backend and scripting environments, including:

- FastAPI applications
- Flask services
- CLI tools
- Data pipelines
- Utility scripts

The AI operates primarily on Python source files while using other files as contextual references.

Website Builder Workspace

Designed for full frontend development, supporting:

- HTML, CSS, JavaScript
- TypeScript and React
- JSX and TSX
- Tailwind CSS
- Multi-page and component-based architectures

Additional capabilities include:

- Responsive design generation
- Image and asset resolution
- Live preview rendering
- Automatic preview error detection and repair

Each workspace uses a specialized system prompt, ensuring that AI behavior is tailored to the development environment.

Both frontend and backend enforce workspace restrictions, with the backend acting as the final authority.

2.3.2 AI-Assisted Coding

The AI assistant provides multiple capabilities:

Code Generation

- Creation of new files, components, and modules
- Generation of routes, utilities, and layouts
- Initialization of project structures

Code Modification

- Updating existing files based on natural language instructions
- Identifying correct implementation locations
- Applying structured changes

Code Explanation

- Explaining functions, components, and logic
- Interpreting error messages
- Describing architecture and flow

Context-Aware Assistance

The AI uses project-aware context selection based on:

- File names and relevance
- Import relationships
- Open and recent files
- Current file content
- Conversation history

Structured File Operations

AI outputs are constrained to:

- create
- update
- delete

These are parsed and executed through controlled backend logic.

Undo Support

Before applying changes, the system stores sufficient metadata to reverse operations:

- Restore modified files
- Recreate deleted files
- Remove newly created files

Automatic Verification and Repair

For website projects, the system detects issues such as:

- Missing imports
- Broken entry points
- Invalid assets
- Structural inconsistencies

If detected, the system can trigger automated repair via AI.

2.3.3 Real-Time Editing

Monaco Editor Integration

Features include:

- Syntax highlighting
- Language awareness
- Multi-tab editing
- Word wrapping
- Automatic layout

Auto-Save Mechanism

- Dirty state tracking
- Debounced save requests
- Backend synchronization
- Cache invalidation

Streaming AI Responses

WebSocket-based streaming provides:

- Progressive response rendering
- Token-by-token updates
- Improved user experience

Flow:

- start → delta → done
- Backend applies changes post-generation

Live Website Preview

- Rendered inside sandboxed iframe
- Supports static and React apps
- Uses esbuild-wasm for TSX compilation
- Handles runtime errors and auto-refresh

Collaboration Limitation

Currently, the system does not support multi-user collaboration.

Future implementation would require:

- CRDT or OT systems
- Redis Pub/Sub
- Presence tracking
- Conflict resolution

2.3.4 Project-Based Isolation

Ownership Model

Each project is tied to a specific user, ensuring clear ownership boundaries.

Access Control

Before accessing data, the backend validates:

- JWT authenticity
- User identity
- Project existence
- Ownership relationship

Isolated AI Context

AI operates only within the active project scope, preventing:

- Data leakage between projects
- Cross-project contamination

Isolated Chat History

Each project maintains independent chat sessions, ensuring contextual consistency.

Unique File Paths

MongoDB enforces unique paths per project, avoiding duplication.

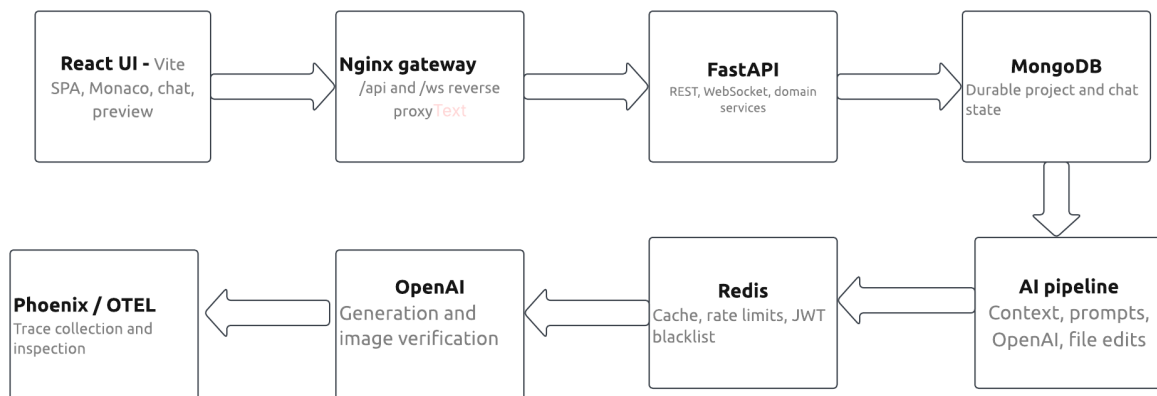
Safe Deletion

Deleting a project removes all associated:

- Files
- Folders
- Chat data
- Cached state

This prevents orphaned data and maintains system integrity.

Runtime request flow



Area	Current implementation
Architecture	Layered FastAPI + React + Mongo + Redis
AI depth	Project-grounded context, routing, edits, repair
RAG	Import/path/keyword retrieval;with embeddings
Data modeling	Separate project, file, folder, chat entities
Versioning	Sequential versioning (v1/v2/..)

Real-time	Token streaming over WebSocket
Security	JWT, bcrypt, ownership, validation, guardrails
Microservices	Modular monolith; five infrastructure containers
Scalability	Async I/O and cache; multithread queue
Testing	AI pipeline unit tests

Summary

The system combines structured architecture, context-aware AI, and real-time interaction to deliver a unified development experience. It balances rapid development with strong architectural foundations, enabling future scalability without sacrificing current usability.